

Assignment #C: 图 (2/4)

Updated 2329 GMT+8 Nov 24, 2025

2025 fall, Compiled by 窦兆文 元培学院

说明：

1. 解题与记录：

对于每一个题目，请提供其解题思路（可选），并附上使用Python或C++编写的源代码（确保已在OpenJudge，Codeforces，LeetCode等平台上获得Accepted）。请将这些信息连同显示“Accepted”的截图一起填写到下方的作业模板中。（推荐使用Typora <https://typoraio.cn> 进行编辑，当然你也可以选择Word。）无论题目是否已通过，请标明每个题目大致花费的时间。

2. **提交安排：**提交时，请首先上传PDF格式的文件，并将.md或.doc格式的文件作为附件上传至右侧的“作业评论”区。确保你的Canvas账户有一个清晰可见的本人头像，提交的文件为PDF格式，并且“作业评论”区包含上传的.md或.doc附件。

3. **延迟提交：**如果你预计无法在截止日期前提交作业，请提前告知具体原因。这有助于我们了解情况并可能为你提供适当的延期或其他帮助。

请按照上述指导认真准备和提交作业，以保证顺利完成课程要求。

1. 题目

M909.蛇梯棋

bfs, <https://leetcode.cn/problems/snakes-and-ladders/>

思路：

棋盘编号是"转行交替方式"，需要将编号转换为坐标

每次可以移动1-6步（模拟骰子）

如果落在蛇或梯子上，会传送到目的地

使用BFS找最短路径

代码：

```

class Solution(object):
    def snakesAndLadders(self, board):
        """
        :type board: List[List[int]]
        :rtype: int
        """
        n = len(board)
        def get_position(square):
            square -= 1
            row = n - 1 - square // n
            col = square % n
            if (n - 1 - row) % 2 == 1:
                col = n - 1 - col
            return row, col
        from collections import deque
        queue = deque([1])
        visited = {1}
        steps = 0

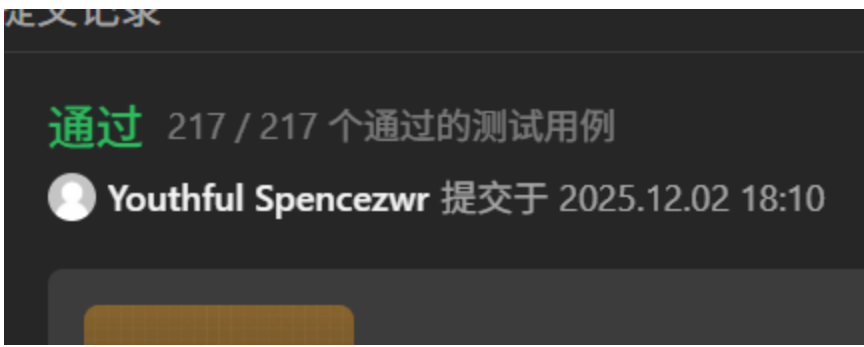
        while queue:
            size = len(queue)
            for _ in range(size):
                curr = queue.popleft()
                if curr == n * n:
                    return steps
                for i in range(1, 7):
                    next_square = curr + i
                    if next_square > n * n:
                        break
                    row, col = get_position(next_square)
                    if board[row][col] != -1:
                        next_square = board[row][col]
                    if next_square not in visited:
                        visited.add(next_square)
                        queue.append(next_square)

            steps += 1

        return -1

```

代码运行截图（至少包含有"Accepted"）



sy382: 有向图判环 中等

dfs, topological sort, <https://sunnywhy.com/sfbj/10/3/382>

思路:

这个问题可以用 DFS（深度优先搜索）来解决:

标记访问状态: 使用三种状态

未访问 (0)

正在访问 (1): 当前DFS路径上的节点

已完成访问 (2): 已经完成DFS的节点

检测环: 如果在DFS过程中遇到状态为"正在访问"的节点, 说明存在环

优化: 如果节点已经"完成访问"且没有环, 可以直接跳过

代码:

```

def has_cycle():
    n, m = map(int, input().split())
    graph = [[] for _ in range(n)]
    for _ in range(m):
        u, v = map(int, input().split())
        graph[u].append(v)
    state = [0] * n

    def dfs(node):
        if state[node] == 1:
            return True
        if state[node] == 2:
            return False

        state[node] = 1

        for neighbor in graph[node]:
            if dfs(neighbor):
                return True

        state[node] = 2
        return False
    for i in range(n):
        if state[i] == 0:
            if dfs(i):
                print("Yes")
                return

    print("No")

has_cycle()

```

代码运行截图（至少包含有"Accepted"）

完美通过

100% 数据通过测试 [详情](#)

运行时长: 0 ms

M28046: 词梯

bfs, <http://cs101.openjudge.cn/practice/28046/>

思路：

构建无向图（邻接表）

使用BFS从起始单词开始搜索

记录每个单词的前驱，以便回溯路径

找到目标单词后回溯构建完整路径

代码：

```

from collections import deque, defaultdict

def build_graph_optimized(words):
    pattern_dict = defaultdict(list)
    for idx, word in enumerate(words):
        for i in range(len(word)):
            pattern = word[:i] + '*' + word[i+1:]
            pattern_dict[pattern].append(idx)
    graph = [[] for _ in range(len(words))]
    for indices in pattern_dict.values():
        for i in range(len(indices)):
            for j in range(i + 1, len(indices)):
                graph[indices[i]].append(indices[j])
                graph[indices[j]].append(indices[i])

    return graph

def bfs_word_ladder(words, graph, start_word, end_word):
    try:
        start_idx = words.index(start_word)
        end_idx = words.index(end_word)
    except ValueError:
        return None

    queue = deque([start_idx])
    visited = [False] * len(words)
    parent = [-1] * len(words)

    visited[start_idx] = True

    while queue:
        curr = queue.popleft()

        if curr == end_idx:
            path = []
            node = end_idx
            while node != -1:
                path.append(words[node])
                node = parent[node]
            return path[::-1]

        for neighbor in graph[curr]:
            if not visited[neighbor]:

```

```

        visited[neighbor] = True
        parent[neighbor] = curr
        queue.append(neighbor)

    return None

def main():
    n = int(input())
    words = []
    for _ in range(n):
        words.append(input().strip())

    start_word, end_word = input().strip().split()
    graph = build_graph_optimized(words)
    path = bfs_word_ladder(words, graph, start_word, end_word)

    if path:
        print(' '.join(path))
    else:
        print("NO")

if __name__ == "__main__":
    main()

```

代码运行截图（至少包含有"Accepted"）

#51103388提交状态

状态: Accepted

— ... —

M433.最小基因变化

bfs, <https://leetcode.cn/problems/minimum-genetic-mutation/description/>

思路：

使用BFS找最短路径（每次变化算一步）

每次只能改变一个字符，且变化后的基因必须在基因库中

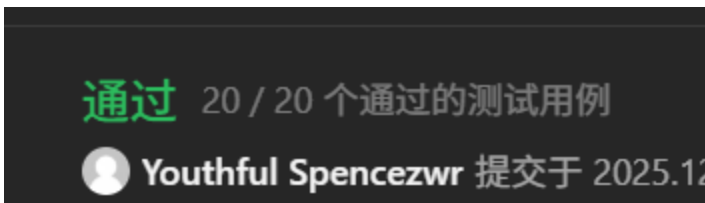
可以尝试将每个位置改为 'A', 'C', 'G', 'T' 四种可能

代码

```
class Solution(object):
    def minMutation(self, startGene, endGene, bank):
        from collections import deque
        if endGene not in bank:
            return -1
        bank_set = set(bank)
        queue = deque([(startGene, 0)])
        visited = {startGene}
        genes = ['A', 'C', 'G', 'T']

        while queue:
            current, steps = queue.popleft()
            if current == endGene:
                return steps
            for i in range(len(current)):
                for gene in genes:
                    if gene == current[i]:
                        continue
                    new_gene = current[:i] + gene + current[i+1:]
                    if new_gene in bank_set and new_gene not in visited:
                        visited.add(new_gene)
                        queue.append((new_gene, steps + 1))
        return -1
```

代码运行截图（至少包含有"Accepted"）



M05443: 兔子与樱花

Dijkstra, <http://cs101.openjudge.cn/practice/05443/>

思路：

使用 Dijkstra 算法 + 路径记录：

构建图（邻接表或邻接矩阵）

对每个查询，使用 Dijkstra 找最短路径

记录前驱节点以便回溯完整路径

代码

```

import heapq
from collections import defaultdict

def dijkstra_with_path(graph, start, end, places):
    if start not in places or end not in places:
        return None, None

    distances = {place: float('inf') for place in places}
    distances[start] = 0
    parent = {place: None for place in places}
    pq = [(0, start)]
    visited = set()

    while pq:
        curr_dist, curr_node = heapq.heappop(pq)

        if curr_node in visited:
            continue

        visited.add(curr_node)
        if curr_node == end:
            break
        for neighbor, weight in graph[curr_node]:
            if neighbor not in visited:
                new_dist = curr_dist + weight
                if new_dist < distances[neighbor]:
                    distances[neighbor] = new_dist
                    parent[neighbor] = curr_node
                    heapq.heappush(pq, (new_dist, neighbor))
    if distances[end] == float('inf'):
        return None, None

    path = []
    node = end
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()

    return path, distances[end]

def get_path_with_distances(graph, path):
    result = []

```

```

for i in range(len(path) - 1):
    curr = path[i]
    next_node = path[i + 1]
    for neighbor, weight in graph[curr]:
        if neighbor == next_node:
            result.append((curr, weight, next_node))
            break
return result

def main():
    P = int(input())
    places = set()
    for _ in range(P):
        place = input().strip()
        places.add(place)
    Q = int(input())
    graph = defaultdict(list)
    for _ in range(Q):
        line = input().strip().split()
        place1 = line[0]
        place2 = line[1]
        distance = int(line[2])
        graph[place1].append((place2, distance))
        graph[place2].append((place1, distance))
    R = int(input())
    for _ in range(R):
        line = input().strip().split()
        start = line[0]
        end = line[1]
        path, total_dist = dijkstra_with_path(graph, start, end, places)

        if path is None:
            print("No path found")
        else:
            path_details = get_path_with_distances(graph, path)
            output = []
            for i, (curr, dist, next_node) in enumerate(path_details):
                if i == 0:
                    output.append(curr)
                output.append(f"->({dist})->")
                output.append(next_node)

            print(''.join(output))

```

```
if __name__ == "__main__":  
    main()
```

代码运行截图（至少包含有"Accepted"）

#51103422提交状态

状态: Accepted

M28050: 骑士周游

dfs, <http://cs101.openjudge.cn/practice/28050/>

思路:

使用回溯算法 + Warnsdorff启发式优化:

从起点开始DFS搜索

记录访问过的格子

如果访问了所有格子, 返回成功

优化: 优先选择下一步可走位置较少的格子 (Warnsdorff规则)

代码:

```

def knight_tour_optimized(n, sr, sc):
    moves = [
        (-2, -1), (-2, 1), (-1, -2), (-1, 2),
        (1, -2), (1, 2), (2, -1), (2, 1)
    ]
    board = [[-1] * n for _ in range(n)]

    def is_valid(x, y):
        return 0 <= x < n and 0 <= y < n and board[x][y] == -1

    def count_unvisited_neighbors(x, y):
        count = 0
        for dx, dy in moves:
            if is_valid(x + dx, y + dy):
                count += 1
        return count

    def get_next_moves(x, y):
        next_moves = []
        for dx, dy in moves:
            next_x, next_y = x + dx, y + dy
            if is_valid(next_x, next_y):
                accessibility = count_unvisited_neighbors(next_x, next_y)
                next_moves.append((accessibility, next_x, next_y))
        next_moves.sort()
        return [(x, y) for _, x, y in next_moves]

    def dfs(x, y, move_count):
        board[x][y] = move_count
        if move_count == n * n - 1:
            return True
        for next_x, next_y in get_next_moves(x, y):
            if dfs(next_x, next_y, move_count + 1):
                return True
        board[x][y] = -1
        return False

    if dfs(sr, sc, 0):
        return "success"
    else:
        return "fail"

n = int(input())

```

```
sr, sc = map(int, input().split())
print(knight_tour_optimized(n, sr, sc))
```

代码运行截图（至少包含有"Accepted"）



2. 学习总结和个人收获

如果发现作业题目相对简单，有否寻找额外的练习题目，如“数算2025fall每日选做”、LeetCode、Codeforces、洛谷等网站上的题目。

图论算法的核心在于**选对遍历方式**（BFS求最短、DFS求存在性、回溯求遍历）和**善用启发式剪枝**（如Warnsdorff规则），看似复杂的问题本质都是在图上做搜索，关键是如何高效地搜。